gemstone-py Parity

gemstone-py Parity Notes

`gemstone-js` intentionally mirrors the practical parts of `gemstone-py` while keeping a JavaScript-first async API.

For the shorter product comparison and batch estimate, see gemstone-js vs gemstone-py.

Current Comparison

`gemstone-js` is now close to `gemstone-py` for core database work: sessions, raw OOP handling, persistent roots, dictionaries, ordered collections, query helpers, migrations, bootstrap, ObjectLog, inspection, code generation, benchmarks, package verification, source/runtime API contract checks, and optional native GCI access are all represented. It now also has the gemstone-py-style bulk selector-send helpers for repeated and mixed raw-OOP calls.

The packaged `gemstone-js-compare` command turns this comparison into a stable CLI report. JSON output includes `$schema: "./schemas/comparison-report.schema.json"` and `schema_version: 1` so editor panels and release tooling can validate it:

```
gemstone-js-compare gemstone-py --scorecard
gemstone-js-compare --target gemstone-py --view scorecard
gemstone-js-compare --target gemstone-py --scope beta --view totals
gemstone-js-compare gemstone-rs --batches
gemstone-js-compare gemstone-py --scorecard --markdown
gemstone-js-compare all --totals --json
gemstone-js-compare --target gemstone-py --view totals --json --assert-total-batches 6 --assert-hours-max
gemstone-js-compare --target gemstone-py --scope beta --view totals --json --assert-total-batches 1 --asse
gemstone-js-compare --target gemstone-py --view totals --max-total-batches 6 --max-hours-max 72 --quiet
gemstone-js-compare --target gemstone-py --scope beta --view totals --max-total-batches 1 --max-hours-max
gemstone-js-compare gemstone-py --scorecard --json --output comparison-report.json
```

`gemstone-py` is still the more mature application stack. It has synchronous and asynchronous Python APIs, Flask/Django/FastAPI/Litestar integrations, larger runnable examples, destructive live-test workflows, benchmark workflow history, and a VS Code workbench. `gemstone-js` is stronger where TypeScript and npm distribution matter: typed generated wrappers, JSON Schemas for manifests/artifacts, explicit async handles, Node/Deno/Bun runtime boundaries, package self-contract checks, npm provenance validation, native prebuild artifact verification, and the native session-thread spike.

Aligned Surface

- Environment-based session configuration and explicit login/logout lifecycle. The JavaScript env names now accept the Pharo bridge aliases `GS_USER`, `GS_PASS`, `GS_NETLDI_HOST`, `GS_NETLDI_NAME_OR_PORT`, and `GS_SERVICE` in addition to the canonical `GS_USERNAME`, `GS_PASSWORD`, `GS_HOST`, `GS_NETLDI`, and `GS_GEM_SERVICE` names.
- Session pools with warmup, wait accounting, reset-aware release, validation queries, custom health checks, idle-timeout eviction, and max-age/max-use recycling, plus provider-style snapshots, lifecycle events, metrics, and spans.
- Framework-neutral request/transaction scopes for lazy session acquisition, commit-on-success, abort-on-error/status, pool release, and owned-session logout.

- Express, Fastify, Fetch API, and Hono adapters wired through the shared request-scope lifecycle with configurable transaction and response-status policy, plus committed example services for each adapter.
- Packaged examples are discoverable through `gemstone-js-examples`, mirroring gemstone-py's installed example catalog pattern in JavaScript form. The catalog now includes quickstart, data helper, query, migration, ObjectLog, codegen, web adapter examples, and a dependency-free browser explorer, with `--kind`, `--commands`, and guided `--plan` views for tooling and focused browsing.
- Raw execute/eval, value marshalling, typed object handles, and managed export-set handles.
- Batched raw selector sends through `bulkPerformOop()` and mixed-call batches through `bulkPerformCallsOop()`, with value-returning variants and `performMany*()`/`performCalls*()` aliases matching gemstone-py's bulk perform workflow. JavaScript-specific `bulkPerformWith()` and `bulkPerformCallsWithOop()` variants batch the same send patterns while first marshalling JavaScript strings, numbers, arrays, dictionaries, and handles through the normal session conversion path. Object-returning batch helpers such as `bulkPerformObjects()` and `bulkPerformCallsObjectsWith()` retain the returned OOPs as typed handles.
- Explicit opt-in value converter registry with named converter lookup, to-OOP/from-OOP helpers, batch conversion, and built-in ISO-string `Date` conversion. Class-instance-to-dictionary helpers mirror gemstone-py's explicit `dataclass_to_dict()` persistence boundary.
- Class-side sends through an explicit class reference object.
- Persistent roots for `UserGlobals`, `Globals`, `Published`, and `SessionMethods`.
- `GStore`-style named JSON key/value stores under `UserGlobals.GStoreRoot`, with bounded store listing, direct readback, and transaction snapshot guards.
- GemStone-side bootstrap audit/source/command helpers for the persistence roots used by the libraries.
- Local setup diagnostics through `gemstone-js-doctor`, covering runtime, environment variables, GCI library discovery, optional native-package availability, JSON output, and opt-in live login/eval.
- Module-style migrations with dependency planning, status/current reads, upgrade/downgrade execution, checksum validation, advisory locks, recorded dry-runs, and a command-line runner.
- Transaction retry, nested transaction, and commit-conflict diagnostic helpers, adapted to JavaScript's async callback style.
- Benchmark report generation for offline `gci` and opt-in live persistence suites, plus baseline comparison, metadata-based baseline selection, baseline manifest registration/replacement, pruning, threshold enforcement, and command-line wrappers for saved report artifacts.
- `StringKeyValueDictionary` helpers with bounded key/item/value-list readback via `KeyedReadbackOptions`/`DictionaryR` pick, require, replace, clear, nested dictionary, raw OOP, and object-handle variants.
- Global and `PersistentRoot` helpers with bounded key/item/value-list readback through `KeyedReadbackOptions`, nullable/required access, raw OOP access, object handles, nested dictionaries, batch setters, and remove/delete variants.
- `OrderedCollection` wrappers for persistent ordered sequences, with explicit async value/raw/object accessors in JavaScript instead of Python magic methods.
- Reduced-conflict wrappers for `RcCounter`, `RcKeyValueDictionary`, and `RcQueue`, including the gemstone-py-style `RCCounter`, `RCHash`, and `RCQueue` aliases plus session factory helpers.
- Collection helpers for search, first/find, count/exists, bounded pages, iteration, mutation, and equality indexes.
- Object inspection, bounded recursive object dumps, direct print-string helpers, class descriptions, and the `gemstone-js-inspect` command.
- ObjectLog entry writes, options-driven bounded batched fetch parsing, tail/latest reads, GemStone-side level filters, latest-by-level reads, count/presence checks, summary/format helpers, size, level-scoped clear, and single/bulk delete helpers.
- Observability hooks, framework adapters, code generation, package verification, public export contract checks, installed-artifact API/bin/schema/release-helper probes, native prebuild artifact verification, and opt-in live smoke coverage with offline live-smoke guard scripts.

JavaScript-Specific Choices

- `Session` is async-first because Node must eventually move GCI work onto a dedicated native session thread.
- Unknown object results stay explicit as raw `Oop` values or retained `TypedOop<T>` handles; JavaScript does not use gemstone-py's dynamic attribute dispatch proxy.
- Future JavaScript object mapping should stay explicit: a mapping manifest schema can generate async

`*Ref` classes around `TypedOop<T>`, repository helpers returning typed refs, and bounded snapshot/dictionary helpers instead of synchronous property dispatch. The current `mappedObject()` helper is the opt-in transparent layer for async property-style selector methods.
- GemStone OOPs are represented as branded `bigint` values in TypeScript, while the native Node boundary uses decimal strings to avoid 64-bit precision loss.
- Root helper names use camelCase (`sessionMethods`) rather than Python's `session_methods`.

Still Python-Only

- The `gemstone-py` VS Code workbench and mature codegen explorer. The JavaScript package now has a small browser explorer example, but a VS Code extension should wait until the package API and native session-thread model settle further.
- Python's dual sync/async public API. JavaScript is intentionally async-first, so parity here should mean equivalent workflows rather than matching sync calls.
- Python's broader framework/example ecosystem, especially Django plus the larger Flask/webstack examples.

Remaining Work Estimate

`gemstone-js` is past the broad parity phase. The remaining work is mostly production hardening and ecosystem polish:

The local beta hardening buckets are now covered: the Node worker backend is selectable from `gemstone-js`, live smoke covers generated wrappers and larger query paths, package checks validate installed JS/native artifacts, and the beta docs cover quickstart, generated wrappers, troubleshooting, and the support boundary.

That leaves one focused validation batch from a conservative beta: run the release candidate from clean installed artifacts across the supported native and live path, then review the produced metadata before publishing. The packaged comparison CLI tracks that conservative beta lane explicitly with `gemstone-js-compare --target gemstone-js --scope beta --view totals`, which currently reports 1 batch and roughly 4-8 hours.

The broader ecosystem comparison uses a fuller product-parity yardstick. Under that view, `gemstone-js-compare all --batches` reports 12 total batches across the JavaScript and Rust catch-up tracks. The Rust/Python total is 6 batches, reported by `gemstone-js-compare gemstone-rs --totals`.

Batch Roadmap

1. Release candidate validation: run `npm run verify`, `npm run native-install:check`, and `GS_RUN_LIVE=1 GS_NATIVE_SESSION_WORKER=1 npm run test:live` from clean installed artifacts, then inspect CI tarballs, checksums, provenance metadata, and native prebuild artifacts before publishing.

The next non-beta product batch should be connector-inspired object mapping: mapping manifest schema, generated `*Ref` classes, repository helpers, bounded snapshot/dictionary helpers, and Explorer/VS Code mapping views over committed generated files.

The VS Code extension is still a later track. It is valuable, but the JavaScript package and native session-thread behavior should stabilize first so the extension does not freeze unstable API assumptions into editor workflows.